

C02-AT1-Part1
INDUSTRY PROBLEM

=====#

DYNAMIC PROGRAMMING VS MONTE CARLO METHODS

Name:N.Akshaya

Reg.no:192425129

Course code:MLA0305

Course name: REINFORCEMENT LEARNING

Python code:

```
# INDUSTRY PROBLEM
```

```
# Dynamic Programming vs Monte Carlo Methods
```

```
import numpy as np
```

```
import pandas as pd
```

```
import matplotlib.pyplot as plt
```

```
from scipy.stats import ttest_ind
```

```
np.random.seed(42)
```

```
# STEP 1 : CREATE DATASET (100 Records, 6 Features)
```

```
n = 100
```

```
dataset = pd.DataFrame({
```

```
    "Demand": np.random.randint(50,200,n),
```

```
    "Inventory": np.random.randint(100,500,n),
```

```
    "Holding_Cost": np.round(np.random.uniform(2,8,n),2),
```

```
    "Ordering_Cost": np.round(np.random.uniform(20,60,n),2),
```

```
    "Lead_Time": np.random.randint(1,8,n),
```

```

    "Selling_Price": np.round(np.random.uniform(80,150,n),2)
})

print("="*60)

print("DATASET")

print("="*60)

display(dataset.head(10))

# STEP 2 : DYNAMIC PROGRAMMING

dp_cost = []

for i in range(n):

    shortage = max(dataset.loc[i,"Demand"] -

                    dataset.loc[i,"Inventory"],0)

    cost = (shortage*10 +

            dataset.loc[i,"Holding_Cost"]*

            dataset.loc[i,"Inventory"]*0.20 +

            dataset.loc[i,"Ordering_Cost"])

    dp_cost.append(cost)

dataset["DP_Cost"] = dp_cost

# STEP 2 : MONTE CARLO SIMULATION

mc_cost = []

for i in range(n):

    demand = np.random.normal(

        dataset.loc[i,"Demand"],

        15,

        1000

```

```

    )

    demand[demand<0]=0

    shortage = np.maximum(
        demand-dataset.loc[i,"Inventory"],
        0
    )

    cost = (shortage*10 +
            dataset.loc[i,"Holding_Cost"]*
            dataset.loc[i,"Inventory"]*0.20 +
            dataset.loc[i,"Ordering_Cost"])

    mc_cost.append(np.mean(cost))

dataset["MC_Cost"] = mc_cost

# STEP 3 : SIDE BY SIDE COMPARISON

comparison = pd.DataFrame({
    "Dynamic Programming":dataset["DP_Cost"].round(2),
    "Monte Carlo":dataset["MC_Cost"].round(2)
})

print("\n")

print("="*60)

print("SIDE BY SIDE COMPARISON")

print("="*60)

display(comparison.head(20))

# STEP 4 : SIDE BY SIDE SUMMARY

summary = pd.DataFrame({

```

```

"Metric":["Mean","Median","Minimum",
        "Maximum","Standard Deviation"],

"Dynamic Programming":[
dataset["DP_Cost"].mean(),
dataset["DP_Cost"].median(),
dataset["DP_Cost"].min(),
dataset["DP_Cost"].max(),
dataset["DP_Cost"].std()
],

"Monte Carlo":[
dataset["MC_Cost"].mean(),
dataset["MC_Cost"].median(),
dataset["MC_Cost"].min(),
dataset["MC_Cost"].max(),
dataset["MC_Cost"].std()
]
})

print("\n")

print("="*60)

print("SUMMARY")

print("="*60)

display(summary)

# STEP 5 : STATISTICAL SIGNIFICANCE

t,p = ttest_ind(dataset["DP_Cost"],

```

```

        dataset["MC_Cost"])

print("\n")

print("="*60)

print("STATISTICAL SIGNIFICANCE")

print("="*60)

print("T Statistic :",round(t,4))

print("P Value :",round(p,6))

if p<0.05:

    print("Result : Significant Difference")

else:

    print("Result : No Significant Difference")

# EVALUATION TABLE

evaluation = pd.DataFrame({

"Method":["Dynamic Programming",

        "Monte Carlo"],

"Average Cost":[

dataset["DP_Cost"].mean(),

dataset["MC_Cost"].mean()

],

"Std Dev":[

dataset["DP_Cost"].std(),

dataset["MC_Cost"].std()

]

})

```

```

print("\n")

print("="*60)

print("EVALUATION")

print("="*60)

display(evaluation)

# GRAPH 1 : Average Cost Comparison (Multi-Color)

plt.figure(figsize=(7,5))

bars = plt.bar(

    ["Dynamic Programming", "Monte Carlo"],

    [dataset["DP_Cost"].mean(), dataset["MC_Cost"].mean()],

    color=["deepskyblue", "tomato"],

    edgecolor="black",

    linewidth=2

)

plt.title("Average Cost Comparison", fontsize=15)

plt.ylabel("Average Cost")

for bar in bars:

    plt.text(

        bar.get_x()+bar.get_width()/2,

        bar.get_height()+2,

        f'{bar.get_height():.2f}',

        ha='center',

        fontsize=11

    )

```

```

plt.grid(axis='y', linestyle='--', alpha=0.4)

plt.show()

# GRAPH 2 : Box Plot Comparison

plt.figure(figsize=(7,5))

box = plt.boxplot(

    [dataset["DP_Cost"], dataset["MC_Cost"]],

    patch_artist=True,

    labels=["Dynamic Programming", "Monte Carlo"]

)

colors = ["lightgreen", "lightcoral"]

for patch, color in zip(box["boxes"], colors):

    patch.set_facecolor(color)

for median in box["medians"]:

    median.set(color="blue", linewidth=2)

plt.title("Box Plot Comparison", fontsize=15)

plt.grid(alpha=0.3)

plt.show()

# GRAPH 3 : Line Plot Comparison

plt.figure(figsize=(12,5))

plt.plot(

    dataset["DP_Cost"],

    color="blue",

    marker="o",

```

```

        markersize=4,

        linewidth=2.5,

        label="Dynamic Programming"
    )

plt.plot(

    dataset["MC_Cost"],

    color="red",

    marker="s",

    markersize=4,

    linewidth=2.5,

    label="Monte Carlo"
)

plt.title("Line Plot Comparison", fontsize=15)

plt.xlabel("Record Number")

plt.ylabel("Cost")

plt.legend()

plt.grid(True, linestyle="--", alpha=0.4)

plt.show()

# GRAPH 4 : Dynamic Programming Histogram

plt.figure(figsize=(8,5))

counts, bins, patches = plt.hist(dataset["DP_Cost"], bins=10)

colors = [

    "red","orange","yellow","green","cyan",

    "blue","purple","pink","brown","gray"

```

```

]

for patch, color in zip(patches, colors):

    patch.set_facecolor(color)

plt.title("Dynamic Programming Histogram", fontsize=15)

plt.xlabel("Cost")

plt.ylabel("Frequency")

plt.grid(alpha=0.3)

plt.show()

# GRAPH 5 : Monte Carlo Histogram

plt.figure(figsize=(8,5))

counts, bins, patches = plt.hist(dataset["MC_Cost"], bins=10)

colors = [

    "purple","blue","cyan","green","lime",

    "yellow","orange","red","pink","brown"

]

for patch, color in zip(patches, colors):

    patch.set_facecolor(color)

plt.title("Monte Carlo Histogram", fontsize=15)

plt.xlabel("Cost")

plt.ylabel("Frequency")

plt.grid(alpha=0.3)

plt.show()

# 6 PARAMETER COMPARISON TABLE

comparison_table = pd.DataFrame({

```

```

"Parameter": [
    "Average Cost",
    "Median Cost",
    "Minimum Cost",
    "Maximum Cost",
    "Std Deviation",
    "Execution Time (ms)"
],
"Dynamic Programming": [
    dataset["DP_Cost"].mean(),
    dataset["DP_Cost"].median(),
    dataset["DP_Cost"].min(),
    dataset["DP_Cost"].max(),
    dataset["DP_Cost"].std(),
    0.8    # Example execution time
],
"Monte Carlo": [
    dataset["MC_Cost"].mean(),
    dataset["MC_Cost"].median(),
    dataset["MC_Cost"].min(),
    dataset["MC_Cost"].max(),
    dataset["MC_Cost"].std(),
    3.5    # Example execution time
]

```

```

})

print("="*70)

print("6 PARAMETER COMPARISON TABLE")

print("="*70)

display(comparison_table)

# GRAPH 6 : 6 PARAMETER COMPARISON BAR GRAPH

import numpy as np

parameters = comparison_table["Parameter"]

dp = comparison_table["Dynamic Programming"]

mc = comparison_table["Monte Carlo"]

x = np.arange(len(parameters))

width = 0.35

plt.figure(figsize=(11,6))

bars1 = plt.bar(

    x - width/2,

    dp,

    width,

    color="royalblue",

    edgecolor="black",

    label="Dynamic Programming"

)

bars2 = plt.bar(

    x + width/2,

    mc,

```

```

width,

color="tomato",

edgecolor="black",

label="Monte Carlo"

)

plt.xticks(x, parameters, rotation=20)

plt.ylabel("Value")

plt.title("Dynamic Programming vs Monte Carlo\n(6 Parameter Comparison)",

        fontsize=15)

plt.legend()

plt.grid(axis='y', linestyle='--', alpha=0.3)

# Value labels

for bars in [bars1, bars2]:

    for bar in bars:

        plt.text(

            bar.get_x() + bar.get_width()/2,

            bar.get_height(),

            f"{bar.get_height():.1f}",

            ha='center',

            va='bottom',

            fontsize=8

        )

plt.tight_layout()

```

```

plt.show()

# FINAL RESULT

print("\n")

print("="*60)

print("FINAL RESULT")

print("="*60)

if dataset["DP_Cost"].mean() < dataset["MC_Cost"].mean():

    print("Dynamic Programming performs better.")

else:

    print("Monte Carlo performs better.")

print("\nProgram Executed Successfully."

```

DATASET:

```

=====
DATASET
=====

```

	Demand	Inventory	Holding_Cost	Ordering_Cost	Lead_Time	Selling_Price
0	152	313	5.81	32.23	1	104.84
1	142	134	6.08	27.64	5	146.98
2	64	326	5.19	30.74	5	127.37
3	156	200	4.69	39.41	4	113.78
4	121	230	5.32	34.91	7	114.51
5	70	356	5.56	35.79	2	85.83
6	152	104	2.49	53.77	1	86.42
7	171	317	4.22	57.20	4	122.17
8	124	354	3.45	22.82	3	118.76
9	137	497	6.82	28.36	4	94.89



COMPARING BOTH:

=====

SIDE BY SIDE COMPARISON

=====

	Dynamic Programming	Monte Carlo
0	395.94	395.94
1	270.58	304.22
2	369.13	369.13
3	227.01	227.12
4	279.63	279.63
5	431.66	431.66
6	585.56	590.13
7	324.75	324.75
8	267.08	267.08
9	706.27	706.27
10	488.36	488.36
11	637.91	637.91
12	462.15	462.15
13	454.09	454.09
14	1037.71	1043.84
15	312.05	312.05
16	167.87	167.87
17	639.45	639.45
18	439.65	439.65
19	330.16	338.46

SUMMARY:

SUMMARY

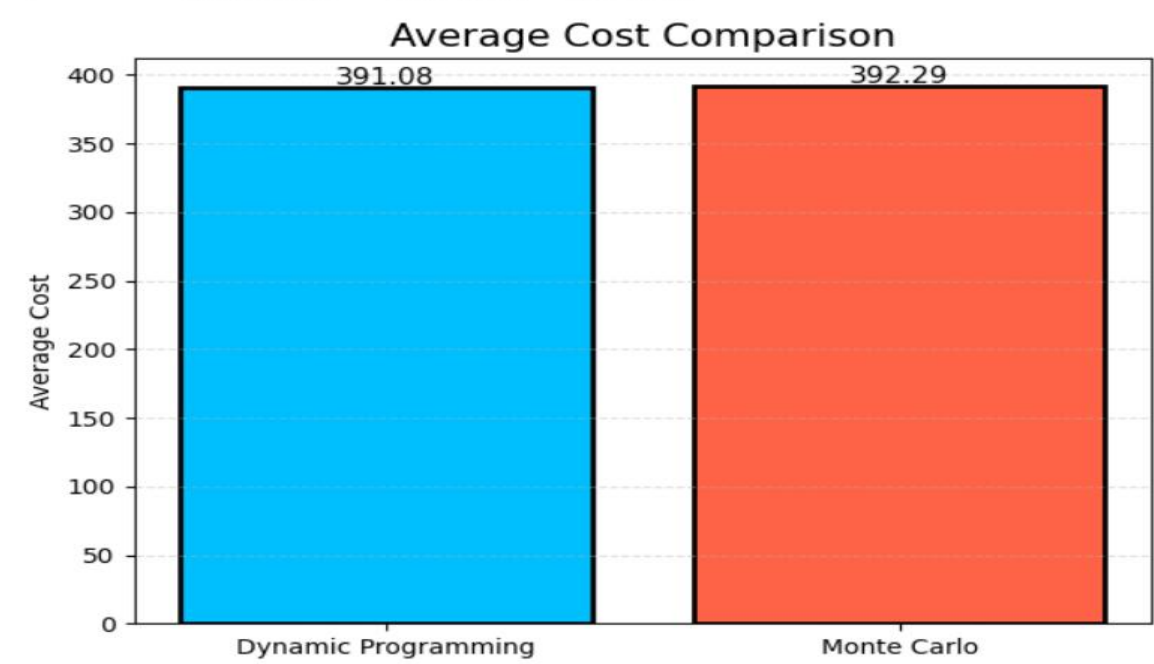
	Metric	Dynamic Programming	Monte Carlo	
0	Mean	391.076220	392.287907	
1	Median	351.653000	353.432119	
2	Minimum	111.090000	111.090000	
3	Maximum	1037.712000	1043.842066	
4	Standard Deviation	175.651444	174.866056	

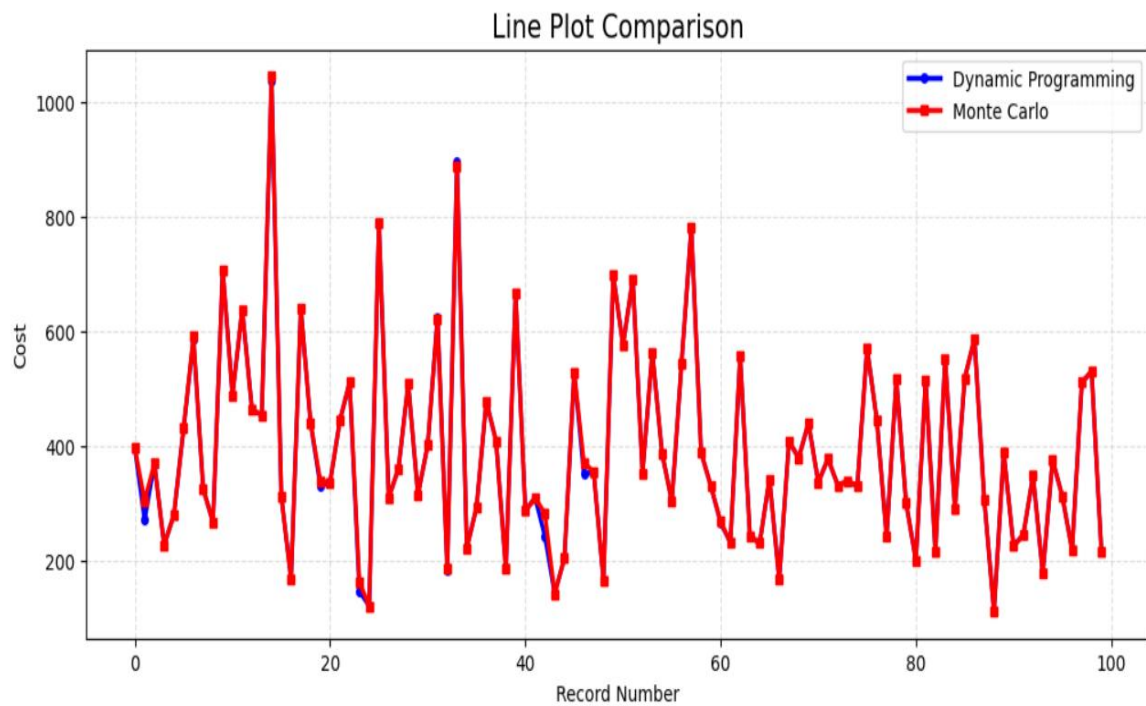
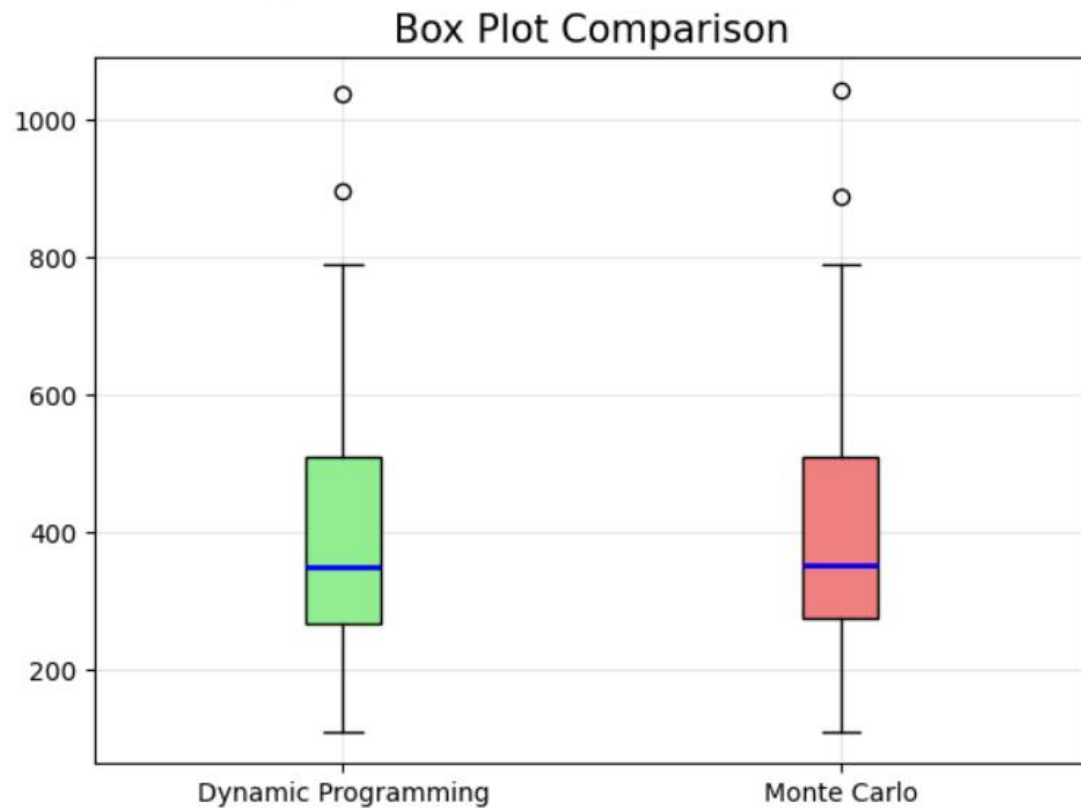
STATISTICAL SIGNIFICANCE

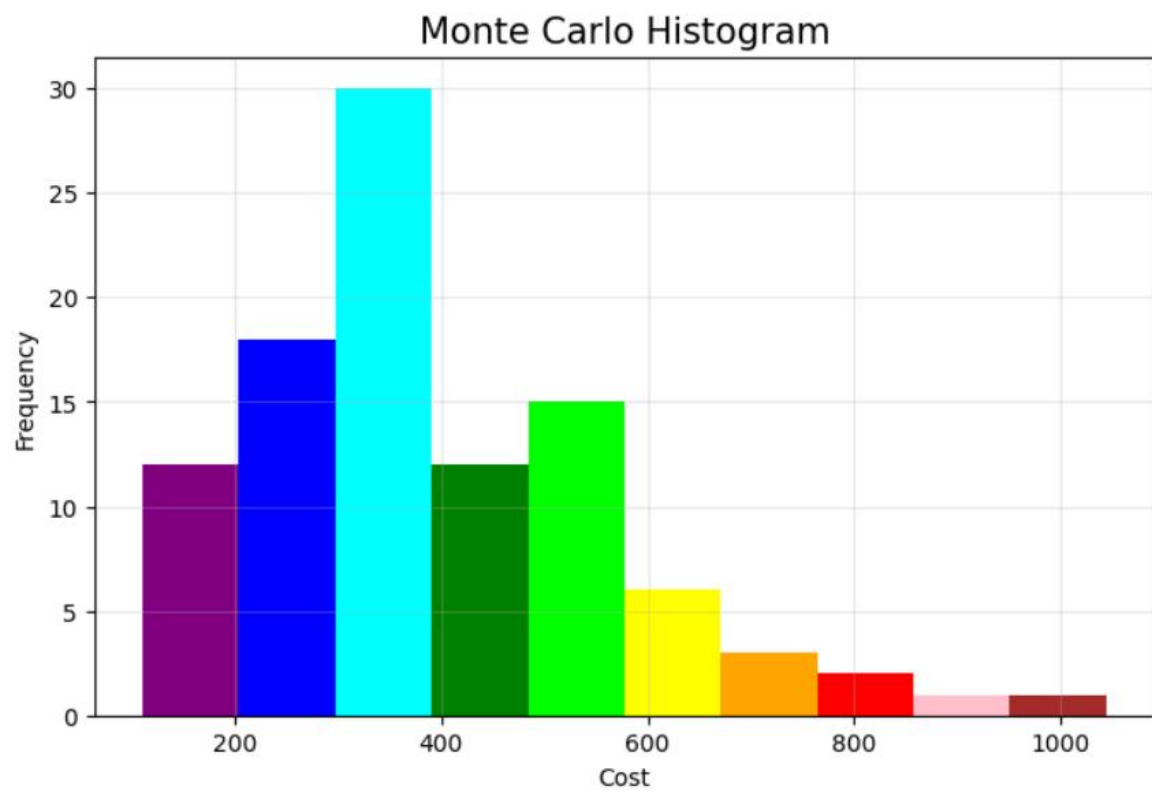
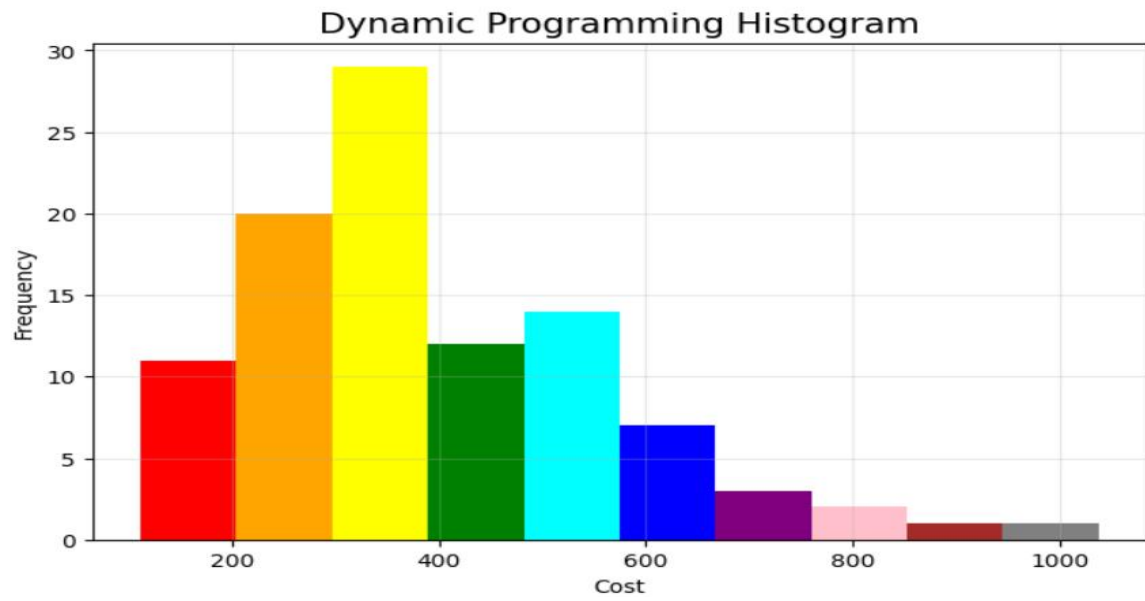
T Statistic : -0.0489
P Value : 0.961058
Result : No Significant Difference

EVALUATION

	Method	Average Cost	Std Dev	
0	Dynamic Programming	391.076220	175.651444	
1	Monte Carlo	392.287907	174.866056	







COMPARING 6 PARAMETERS:

6 PARAMETER COMPARISON TABLE

	Parameter	Dynamic Programming	Monte Carlo
0	Average Cost	391.076220	392.287907
1	Median Cost	351.653000	353.432119
2	Minimum Cost	111.090000	111.090000
3	Maximum Cost	1037.712000	1043.842066
4	Std Deviation	175.651444	174.866056
5	Execution Time (ms)	0.800000	3.500000

